different web frameworks. The most popular is Ruby on Rails, which provides web features along with database connectivity, and a complete grab bag of everything you'd need to build a full-featured web applications.

Rails is great, but it's also a lot to learn. In this book, we're going to show a simpler framework called Sinatra,[13] which is one of the fastest ways to get a basic web application up and running in Ruby.

Here's a minimal Sinatra file:

web/sinatra_01/sinatra.rb

```
require "bundler/inline"

gemfile do
  source "https://rubygems.org"
  gem "sinatra", require: false
  gem "thin"
end

require "sinatra"

get "/" do
  "<h1>Fly me to the moon!</h1>"
end
```

To start a web server, all you need to do is run this file like an ordinary Ruby program.

```
$ ruby sinatra.rb
== Sinatra (v3.0.0) has taken the stage on 4567
for development with backup from Thin
2022-10-15 08:48:20 -0500 Thin web server (v1.8.1 codename Infinite Smoothie)
2022-10-15 08:48:20 -0500 Maximum connections set to 1024
2022-10-15 08:48:20 -0500 Listening on localhost:4567, CTRL+C to stop
```

From there, pointing a web browser at http://localhost:4567 will result in a web page being served, with "Fly me to the moon", just as specified at the end of the code.

How is this working? Logistically, the first ten lines or so of this example are using bundler/inline to install gems as part of the script, which is a useful trick for simple Sinatra scripts. We're pulling in both Sinatra itself, and the Thin web server, when it runs, Sinatra will automatically use Thin if it is available. (You can use other servers, but you need to create a Rackup file to do so.) Because of the way Sinatra decides when to run, we need to explicitly require "sinatra", which means we need Bundler not to require Sinatra, so the line in the gemfile section for Sinatra needs to include require: false.

The actual web serving part is the last three lines. Generically, a Sinatra application is a collection of routes. A route is an HTTP verb, followed by a path, followed by some optional options and then a block. The HTTP verb, path and options tell Sinatra how to match requests to routes, and the block tell Sinatra how to reply when the request matches the route.

In this case, we've defined one route, the HTTP verb is GET, it matches the root path of /, and there are no other options. The return value of the block is the string "<h1>Fly me to the moon!</h1>". And so, if we hit the root route, we get that response. Hitting any other URL within that server triggers a 404 error page, since Sinatra has no routes matching the URL.

13. https://sinatrarb.com